Com S // Cpr E // Math 5250

Numerical Analysis of High-Performance Computing

Instructor: Songting Luo

Lecture 13: Matrix Struct and Finding Eigenvalues

Outline

1. Matrix Struct
2. Finding Eigenvalues

Matrix Struct

Matrix Struct: header file (struct + macros)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.h:

```
#ifndef __MATRIX_H__
#define __MATRIX_H__

typedef struct matrix matrix;
struct matrix
{ int rows; int cols; double* val; };

typedef struct vector vector;
struct vector
{ int size; double* val; };

// Shortcut evaluate functions
#define mget(mat,i,j) mat.val[(i-1)*mat.cols+(j-1)]
#define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
#define vget(vec,i) vec.val[(i-1)]
#define vgetp(vec,i) vec->val[(i-1)]
#define print_matrix(mat) print_matrix_full(mat,#mat);
#define print_vector(vec) print_vector_full(vec,#vec);
#define print_scalar(z) print_scalar_full(z,#z);
```

Matrix Struct: header file (declarations)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.h:

```
// Function declarations
matrix new_matrix(const int rows, const int cols);
void print_matrix_full(const matrix* mat, char* varname);
matrix matrix_add(const matrix* A, const matrix* B);
matrix matrix_sub(const matrix* A, const matrix* B);
matrix matrix_mult(const matrix* A, const matrix* B);
matrix matrix_dot_mult(const matrix* A, const matrix* B);
//
vector new_vector(const int size);
void print_vector_full(const vector* vec, char* varname);
vector vector_add(const vector* x, const vector* y);
vector vector_sub(const vector* x, const vector* y);
double vector_dot_mult(const vector* x, const vector* y);
//
void print_scalar_full(const double* z, char* varname);
//
vector matrix_vector_mult(const matrix* A, const vector* x);
//
vector solve(const matrix* A, const vector* b);

#endif
```

Matrix Struct: source file (create a new matrix)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```c
#include <stdlib.h>
#include <stdio.h>
#include <assert.h>
#include <math.h>
#include "matrix.h"

matrix new_matrix(const int rows, const int cols)
{
   matrix mat;
   mat.rows = rows;
   mat.cols = cols;
   assert(rows>0);
   assert(cols>0);
   mat.val = (double*)malloc(sizeof(double)*rows*cols);

   for (int i=0; i<(rows*cols); i++)
   {  mat.val[i] = 0.0;  }

   return mat;
}
```

Matrix Struct: source file (print matrix)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```c
void print_matrix_full(const matrix* mat, char* varname)
{
   assert(mat->rows>0); assert(mat->cols>0);
   printf("\n %.100s =\n", &varname[1] );
   for(int i=1; i<=mat->rows; i++ )
   {
      printf("  |  ");
      for(int j=1; j<=mat->cols; j++)
      {
         printf("%10.3e",mgetp(mat,i,j));
         if (j<mat->cols) {printf(", ");}
         else {printf(" ");}
      }
      printf("|\n");
   }
   printf("\n");
}
```

```c
#define print_matrix(mat) print_matrix_full(mat,#mat);
```

```c
#define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
```

Matrix Struct: source file (add two matrices)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
matrix matrix_add(const matrix* A, const matrix* B)
{
   const int rows = A->rows;
   const int cols = A->cols;
   assert(rows==B->rows);
   assert(cols==B->cols);
   matrix C = new_matrix(rows,cols);

   for (int i=1; i<=rows; i++)
      for (int j=1; j<=cols; j++)
      {
         mget(C,i,j) = mgetp(A,i,j)+mgetp(B,i,j);
      }

   return C;
}
```

```
#define mget(mat,i,j) mat.val[(i-1)*mat.cols+(j-1)]
#define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
```

Matrix Struct: source file (matrix-matrix product)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
matrix matrix_mult(const matrix* A, const matrix* B)
{
   const int rowsA = A->rows; const int colsA = A->cols;
   const int rowsB = B->rows; const int colsB = B->cols;
   assert(colsA==rowsB);
   matrix C = new_matrix(rowsA,colsB);

   for (int i=1; i<=rowsA; i++)
      for (int j=1; j<=colsB; j++)
         for (int k=1; k<=colsA; k++)
         {
            mget(C,i,j) += mgetp(A,i,k)*mgetp(B,k,j);
         }

   return C;
}
```

```
#define mget(mat,i,j) mat.val[(i-1)*mat.cols+(j-1)]
#define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
```

Matrix Struct: source file (element-by-element product)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
matrix matrix_dot_mult(const matrix* A, const matrix* B)
{
   const int rows = A->rows;
   const int cols = A->cols;
   assert(rows==B->rows);
   assert(cols==B->cols);
   matrix C = new_matrix(rows,cols);

   for (int i=1; i<=rows; i++)
      for (int j=1; j<=cols; j++)
      {
         mget(C,i,j) = mgetp(A,i,j)*mgetp(B,i,j);
      }

   return C;
}
```

```
#define mget(mat,i,j) mat.val[(i-1)*mat.cols+(j-1)]
#define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
```

Matrix Struct: source file (create vector)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
vector new_vector(const int size)
{
    vector vec;
    vec.size = size;
    assert(size>0);
    vec.val = (double*)malloc(sizeof(double)*size);

    for (int i=0; i<(size); i++)
    {  vec.val[i] = 0.0;  }

    return vec;
}
```

Matrix Struct: source file (print vector)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```c
void print_vector_full(const vector* vec, char* varname)
{
   assert(vec->size>0);
   printf("\n");
   printf(" %.100s =\n", &varname[1] );
   printf("  |  ");
   for(int i=1; i<=vec->size; i++ )
   {
      printf("%10.3e",vgetp(vec,i));
      if (i<vec->size) {printf(", ");}
   }
   printf(" |^T\n\n");
}
```

```c
#define print_vector(vec) print_vector_full(vec,#vec);
```

Matrix Struct: source file (add two vectors)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
vector vector_add(const vector* x, const vector* y)
{
   const int size = x->size;
   assert(size==y->size);
   vector z = new_vector(size);

   for (int i=1; i<=size; i++)
   {
      vget(z,i) = vgetp(x,i)+vgetp(y,i);
   }

   return z;
}
```

```
#define vget(vec,i) vec.val[(i-1)]
#define vgetp(vec,i) vec->val[(i-1)]
```

Matrix Struct: source file (dot product of two vectors)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
double vector_dot_mult(const vector* x, const vector* y)
{
   const int size = x->size; assert(size==y->size);

   double z = 0.0;
   for (int i=1; i<=size; i++)
   { z += vgetp(x,i)*vgetp(y,i); }

   return z;
}

void print_scalar_full(const double* z, char* varname)
{
   printf("\n %.100s =\n", &varname[1] );
   printf("    %10.3e \n\n",*z);
}
```

```
#define vget(vec,i) vec.val[(i-1)]
```

```
#define print_scalar(z) print_scalar_full(z,#z);
```

Matrix Struct: source file (matrix-vector multiply)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
vector matrix_vector_mult(const matrix* A, const vector* x)
{
   const int rows = A->rows; const int cols = A->cols;
   const int size = x->size;
   assert(cols==size);
   vector Ax = new_vector(rows);

   for (int i=1; i<=rows; i++)
   {
      double tmp = 0.0;
      for (int j=1; j<=size; j++)
      { tmp += mgetp(A,i,j)*vgetp(x,j); }
      vget(Ax,i) = tmp;
   }

   return Ax;
}
```

```
#define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
#define vget(vec,i) vec.val[(i-1)]
#define vgetp(vec,i) vec->val[(i-1)]
```

Matrix Struct: source file (Gaussian elimination with partial pivoting)

$$A = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix}$$

$$\longrightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ 4 & 3 & 3 & 1 \\ 2 & 1 & 1 & 0 \\ 6 & 7 & 9 & 8 \end{bmatrix} \longrightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ & -1/2 & -3/2 & -3/2 \\ & -3/4 & -5/4 & -5/4 \\ & 7/4 & 9/4 & 17/4 \end{bmatrix}$$

$$\longrightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ & 7/4 & 9/4 & 17/4 \\ & -3/4 & -5/4 & -5/4 \\ & -1/2 & -3/2 & -3/2 \end{bmatrix} \longrightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ & 7/4 & 9/4 & 17/4 \\ & & -2/7 & 4/7 \\ & & -6/7 & -2/7 \end{bmatrix}$$

$$\longrightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ & 7/4 & 9/4 & 17/4 \\ & & -6/7 & -2/7 \\ & & -2/7 & 4/7 \end{bmatrix} \longrightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ & 7/4 & 9/4 & 17/4 \\ & & -6/7 & -2/7 \\ & & & 2/3 \end{bmatrix}$$

Matrix Struct: source file (Gaussian elimination with partial pivoting)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
vector solve(const matrix* A, const vector* b)
{
   const int rows = A->rows; const int cols = A->cols;
   const int size = b->size;
   assert(rows==cols); assert(rows==size);

   vector x = new_vector(rows);

   for (int i=1; i<=(size-1); i++) // LOOP OVER EACH COLUMN
   {
      // Select largest pivot in current column
      int p=i; double maxA = -100.0e0;
      for (int j=i; j<=size; j++)
      {
         double tmp = fabs(mgetp(A,j,i));
         if ( tmp > maxA ){ p = j; maxA = tmp; }
      }

      // See if matrix is singular
      if (maxA <= 1.0e-14)
      { printf(" Cannot invert system\n"); exit(1); }
```

Matrix Struct: source file (Gaussian elimination with partial pivoting)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
        // Pivot (aka interchange rows)
        if (p!=i)
        {
            for (int j=1; j<=size; j++)
            {
                double tmp1  = mgetp(A,i,j);
                mgetp(A,i,j) = mgetp(A,p,j); mgetp(A,p,j) = tmp1;
            }

            double tmp2 = vgetp(b,i);
            vgetp(b,i) = vgetp(b,p); vgetp(b,p) = tmp2;
        }

        // Eliminate below diagonal
        for (int j=(i+1); j<=size; j++)
        {
            double dm = mgetp(A,j,i)/mgetp(A,i,i);
            for (int k=1; k<=size; k++)
            { mgetp(A,j,k) = mgetp(A,j,k) - dm*mgetp(A,i,k); }
            vgetp(b,j) = vgetp(b,j) - dm*vgetp(b,i);
        }
    }
```

Matrix Struct: source file (Gaussian elimination with partial pivoting)

$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
    // Backward substitution
    vget(x,size) = vgetp(b,size)/mgetp(A,size,size);
    for (int j=1; j<=(size-1); j++)
    {
       double sum = 0.0e0;

       for (int k=(size-j+1); k<=size; k++)
       { sum = sum + mgetp(A,size-j,k)*vget(x,k); }

       vget(x,size-j) = (vgetp(b,size-j) - sum)
          /mgetp(A,size-j,size-j);
    }

    return x;
 }
```

Matrix Struct: Makefile

$ISUHPC/lectures/lecture13/codes/matrix/Makfile:

```
# $ISUHPC/lectures/lecture12/codes/matrix/Makefile
CC = gcc
FFLAGS = -O3 -Wall -std=c99
LFFLAG =
OBJECTS = main.o \
matrix.o

.PHONY: clean

main.exe: $(OBJECTS)
        $(CC) $(LFLAGS) $(OBJECTS) -o main.exe

%.o: %.c
        $(CC) $(FFLAGS) -c $<

clean:
        rm -f $(OBJECTS) main.exe
```

Matrix Struct: main function

$ISUHPC/lectures/lecture13/codes/matrix/main.c:

```c
#include <stdlib.h>
#include <stdio.h>
#include "matrix.h"

int main()
{
    // Matrices
    matrix A = new_matrix(5,5);
    matrix B = new_matrix(5,5);

    for(int i=1; i<=5; i++ )
        for (int j=1; j<=5; j++ )
        {
            mget(A,i,j) = -1.0*(i==j)
                + 2.0*(i-1==j) + 2.0*(j-1==i);
            mget(B,i,j) =  2.0*(i==j)
                + 1.0*(i-1==j) + 1.0*(j-1==i);
        }

    // Print matrices
    print_matrix(&A);
    print_matrix(&B);
```

Matrix Struct: main function

$ISUHPC/lectures/lecture13/codes/matrix/main.c:

```
    // Add/Subtract/Multiply matrices
    matrix  Csum = matrix_add(&A,&B); print_matrix(&Csum);
    matrix Cdiff = matrix_sub(&A,&B); print_matrix(&Cdiff);
    matrix Cprod = matrix_mult(&A,&B); print_matrix(&Cprod);
    matrix  Cdot = matrix_dot_mult(&A,&B); print_matrix(&Cdot);

    // Vectors
    vector x = new_vector(5);
    vector y = new_vector(5);

    vget(x,1) = 1.0;  vget(y,1) = 1.0;
    vget(x,2) = 0.0;  vget(y,2) = 2.0;
    vget(x,3) = 1.0;  vget(y,3) = 3.0;
    vget(x,4) = 0.0;  vget(y,4) = 4.0;
    vget(x,5) = 1.0;  vget(y,5) = 5.0;

    // Print vectors
    print_vector(&x);
    print_vector(&y);
```

Matrix Struct: main function

$ISUHPC/lectures/lecture13/codes/matrix/main.c:

```

    // Add/Subtract/Multiply vectors
    vector  zsum = vector_add(&x,&y); print_vector(&zsum);
    vector zdiff = vector_sub(&x,&y); print_vector(&zdiff);
    double  zdot = vector_dot_mult(&x,&y); print_scalar(&zdot);

    // Matrix vector multiply
    vector Ax = matrix_vector_mult(&A,&x);
    print_vector(&Ax);

    // Linear solve via Gaussian elimination
    vector soln = solve(&A,&y);
    print_vector(&soln);
}
```

Matrix Struct: compilation & execution

```
$ make
gcc -O3 -Wall -c main.c
gcc -O3 -Wall -c matrix.c
gcc  main.o matrix.o -o main.exe

$ main.exe

 A =
  |  -1.000e+00,  2.000e+00,  0.000e+00,  0.000e+00,  0.000e+00 |
  |   2.000e+00, -1.000e+00,  2.000e+00,  0.000e+00,  0.000e+00 |
  |   0.000e+00,  2.000e+00, -1.000e+00,  2.000e+00,  0.000e+00 |
  |   0.000e+00,  0.000e+00,  2.000e+00, -1.000e+00,  2.000e+00 |
  |   0.000e+00,  0.000e+00,  0.000e+00,  2.000e+00, -1.000e+00 |

 B =
  |   2.000e+00,  1.000e+00,  0.000e+00,  0.000e+00,  0.000e+00 |
  |   1.000e+00,  2.000e+00,  1.000e+00,  0.000e+00,  0.000e+00 |
  |   0.000e+00,  1.000e+00,  2.000e+00,  1.000e+00,  0.000e+00 |
  |   0.000e+00,  0.000e+00,  1.000e+00,  2.000e+00,  1.000e+00 |
  |   0.000e+00,  0.000e+00,  0.000e+00,  1.000e+00,  2.000e+00 |
```

Matrix Struct: compilation & execution

```
Csum =
 |   1.000e+00,  3.000e+00,  0.000e+00,  0.000e+00,  0.000e+00 |
 |   3.000e+00,  1.000e+00,  3.000e+00,  0.000e+00,  0.000e+00 |
 |   0.000e+00,  3.000e+00,  1.000e+00,  3.000e+00,  0.000e+00 |
 |   0.000e+00,  0.000e+00,  3.000e+00,  1.000e+00,  3.000e+00 |
 |   0.000e+00,  0.000e+00,  0.000e+00,  3.000e+00,  1.000e+00 |

Cdiff =
 |  -3.000e+00,  1.000e+00,  0.000e+00,  0.000e+00,  0.000e+00 |
 |   1.000e+00, -3.000e+00,  1.000e+00,  0.000e+00,  0.000e+00 |
 |   0.000e+00,  1.000e+00, -3.000e+00,  1.000e+00,  0.000e+00 |
 |   0.000e+00,  0.000e+00,  1.000e+00, -3.000e+00,  1.000e+00 |
 |   0.000e+00,  0.000e+00,  0.000e+00,  1.000e+00, -3.000e+00 |

Cprod =
 |   0.000e+00,  3.000e+00,  2.000e+00,  0.000e+00,  0.000e+00 |
 |   3.000e+00,  2.000e+00,  3.000e+00,  2.000e+00,  0.000e+00 |
 |   2.000e+00,  3.000e+00,  2.000e+00,  3.000e+00,  2.000e+00 |
 |   0.000e+00,  2.000e+00,  3.000e+00,  2.000e+00,  3.000e+00 |
 |   0.000e+00,  0.000e+00,  2.000e+00,  3.000e+00,  0.000e+00 |
```

Matrix Struct: compilation & execution

```
Cdot =
  |  -2.000e+00,  2.000e+00,  0.000e+00,  0.000e+00,  0.000e+00 |
  |   2.000e+00, -2.000e+00,  2.000e+00,  0.000e+00,  0.000e+00 |
  |   0.000e+00,  2.000e+00, -2.000e+00,  2.000e+00,  0.000e+00 |
  |   0.000e+00,  0.000e+00,  2.000e+00, -2.000e+00,  2.000e+00 |
  |   0.000e+00,  0.000e+00,  0.000e+00,  2.000e+00, -2.000e+00 |
```

Matrix Struct: compilation & execution

```
x =
 |   1.000e+00,  0.000e+00,  1.000e+00,  0.000e+00,  1.000e+00 |^T

y =
 |   1.000e+00,  2.000e+00,  3.000e+00,  4.000e+00,  5.000e+00 |^T

zsum =
 |   2.000e+00,  2.000e+00,  4.000e+00,  4.000e+00,  6.000e+00 |^T

zdiff =
 |   0.000e+00, -2.000e+00, -2.000e+00, -4.000e+00, -4.000e+00 |^T

zdot =
    9.000e+00

Ax =
 |  -1.000e+00,  4.000e+00, -1.000e+00,  4.000e+00, -1.000e+00 |^T

soln =
 |  -1.606e+00, -3.030e-01,  2.455e+00,  3.030e+00,  1.061e+00 |^T
```

Finding Eigenvalues

Symmetric matrices

Recall: Real matrices, $A \in \mathbb{R}^{n \times n}$, that are symmetric:

$$A = A^T$$

only have real eigenvalues.

Proof: Assume A has a complex eigenvalue: λ, and the corresponding eigenvector, $\vec{v}$:

$$A\mathbf{v} = \lambda \mathbf{v}.$$

Take the complex conjugate of both sides:

$$z = x + iy \quad \Longrightarrow \quad \bar{z} = x - iy,$$

where $\quad i = \sqrt{-1}$:

$$A\bar{\mathbf{v}} = \bar{\lambda}\bar{\mathbf{v}}.$$

Multiply the two statements by $\bar{\mathbf{v}}^T$ and $\mathbf{v}^T$ respectively:

$$\bar{\mathbf{v}}^T A\mathbf{v} = \lambda \bar{\mathbf{v}}^T \mathbf{v} \qquad \textit{and} \qquad \mathbf{v}^T A\bar{\mathbf{v}} = \bar{\lambda}\mathbf{v}^T\bar{\mathbf{v}}$$

$$\Longrightarrow \qquad \mathbf{v}^T A\bar{\mathbf{v}} = \lambda \bar{\mathbf{v}}^T \mathbf{v} \qquad \textit{and} \qquad \mathbf{v}^T A\bar{\mathbf{v}} = \bar{\lambda}\bar{\mathbf{v}}^T\mathbf{v}$$

$$0 = \left(\lambda - \bar{\lambda}\right) \|\mathbf{v}\|^2, \qquad \text{but since} \quad \|\mathbf{v}\| > 0$$

$$\Longrightarrow \qquad \lambda = \bar{\lambda} \qquad \Longrightarrow \qquad \lambda \text{ is real.}$$

Rayleigh Quotient

Problem: Given a vector $\mathbf{x} \in \mathbb{R}^n$ and a real symmetric marix $A \in \mathbb{R}^{n \times n}$, find a real number $\hat{\alpha} \in \mathbb{R}$ such that

$$\|A\mathbf{x} - \hat{\alpha}\mathbf{x}\| = \min_{\alpha \in \mathbb{R}^n} \|A\mathbf{x} - \alpha\mathbf{x}\|.$$

Rayleigh Quotient

Problem: Given a vector $\mathbf{x} \in \mathbb{R}^n$ and a real symmetric marix $A \in \mathbb{R}^{n \times n}$, find a real number $\hat{\alpha} \in \mathbb{R}$ such that

$$\|A\mathbf{x} - \hat{\alpha}\mathbf{x}\| = \min_{\alpha \in \mathbb{R}^n} \|A\mathbf{x} - \alpha\mathbf{x}\|.$$

Solution:

$$\|A\mathbf{x} - \alpha\mathbf{x}\|^2 = (A\mathbf{x} - \alpha\mathbf{x})^T (A\mathbf{x} - \alpha\mathbf{x}) = \left(\mathbf{x}^T A - \alpha\mathbf{x}^T\right)(A\mathbf{x} - \alpha\mathbf{x})$$
$$= \mathbf{x}^T A^2 \mathbf{x} - 2\alpha\mathbf{x}^T A\mathbf{x} + \alpha^2 \mathbf{x}^T\mathbf{x}$$

(concave up quadratic function, unique global min where $d/d\alpha$ is zero)

$$\frac{d}{d\alpha}\left(\mathbf{x}^T A^2 \mathbf{x} - 2\alpha\mathbf{x}^T A\mathbf{x} + \alpha^2 \mathbf{x}^T\mathbf{x}\right) = 0 \qquad \Longrightarrow \qquad 2\left(\alpha\mathbf{x}^T\mathbf{x} - \mathbf{x}^T A\mathbf{x}\right) = 0$$

$$\therefore \quad \hat{\alpha} = R_A(\mathbf{x}) = \frac{\mathbf{x}^T A\mathbf{x}}{\mathbf{x}^T\mathbf{x}} \qquad \text{(Rayleigh Quotient)}$$

Rayleigh Quotient

Problem: Given a vector $\mathbf{x} \in \mathbb{R}^n$ and a real symmetric marix $A \in \mathbb{R}^{n \times n}$, find a real number $\hat{\alpha} \in \mathbb{R}$ such that

$$\|A\mathbf{x} - \hat{\alpha}\mathbf{x}\| = \min_{\alpha \in \mathbb{R}^n} \|A\mathbf{x} - \alpha\mathbf{x}\|.$$

Solution:

$$\|A\mathbf{x} - \alpha\mathbf{x}\|^2 = (A\mathbf{x} - \alpha\mathbf{x})^T (A\mathbf{x} - \alpha\mathbf{x}) = \left(\mathbf{x}^T A - \alpha\mathbf{x}^T\right)(A\mathbf{x} - \alpha\mathbf{x})$$
$$= \mathbf{x}^T A^2 \mathbf{x} - 2\alpha\mathbf{x}^T A\mathbf{x} + \alpha^2 \mathbf{x}^T\mathbf{x}$$

(concave up quadratic function, unique global min where $d/d\alpha$ is zero)

$$\frac{d}{d\alpha}\left(\mathbf{x}^T A^2 \mathbf{x} - 2\alpha\mathbf{x}^T A\mathbf{x} + \alpha^2 \mathbf{x}^T\mathbf{x}\right) = 0 \qquad \Longrightarrow \qquad 2\left(\alpha\mathbf{x}^T\mathbf{x} - \mathbf{x}^T A\mathbf{x}\right) = 0$$

$$\therefore \quad \hat{\alpha} = R_A(\mathbf{x}) = \frac{\mathbf{x}^T A\mathbf{x}}{\mathbf{x}^T\mathbf{x}} \qquad \text{(Rayleigh Quotient)}$$

If $\mathbf{v}$ is an eigenvector of A, then

$$R_A(\mathbf{v}) = \frac{\mathbf{v}^T A\mathbf{v}}{\mathbf{v}^T\mathbf{v}} = \frac{\mathbf{v}^T \lambda\mathbf{v}}{\mathbf{v}^T\mathbf{v}} = \lambda\left(\frac{\mathbf{v}^T\mathbf{v}}{\mathbf{v}^T\mathbf{v}}\right) = \lambda.$$

Estimating the largest eigenvalue

Assume:

$$|\lambda_1| > |\lambda_2| > |\lambda_3| > \cdots > |\lambda_n|$$

Estimating the largest eigenvalue

Assume:

$$|\lambda_1| > |\lambda_2| > |\lambda_3| > \cdots > |\lambda_n|$$

Guess and eigenvector:

$$\mathbf{w}^{(0)} = \sum_{j=1}^{n} \alpha_j \, \mathbf{v}_j$$

where $\mathbf{v}_j$ for $j = 1, 2, \ldots, n$ are the n linearly independent eigenvectors of A.

Estimating the largest eigenvalue

Assume:

$$|\lambda_1| > |\lambda_2| > |\lambda_3| > \cdots > |\lambda_n|$$

Guess and eigenvector:

$$\mathbf{w}^{(0)} = \sum_{j=1}^{n} \alpha_j \, \mathbf{v}_j$$

where $\mathbf{v}_j$ for $j = 1, 2, \ldots, n$ are the n linearly independent eigenvectors of A.

How do we improve this guess?

$$\mathbf{w}^{(1)} = A\mathbf{w}^{(0)} = \sum_{j=1}^{n} \alpha_j \, A\mathbf{v}_j = \sum_{j=1}^{n} \alpha_j \, \lambda_j \mathbf{v}_j$$

$$\mathbf{w}^{(2)} = A\mathbf{w}^{(1)} = A^2\mathbf{w}^{(0)} = \sum_{j=1}^{n} \alpha_j \, A^2\mathbf{v}_j = \sum_{j=1}^{n} \alpha_j \, \lambda_j^2 \mathbf{v}_j$$

$$\mathbf{w}^{(k)} = A\mathbf{w}^{(k-1)} = A^k\mathbf{w}^{(0)} = \sum_{j=1}^{n} \alpha_j \, A^k\mathbf{v}_j = \sum_{j=1}^{n} \alpha_j \, \lambda_j^k \mathbf{v}_j \approx \alpha_1 \, \lambda_1^k \mathbf{v}_1$$

$$\therefore \quad \lambda_1 \approx R_A\left(\mathbf{w}^{(k)}\right)$$

Power Iteration Algorithm

INPUT: $\mathbf{v}^{(0)} \in \mathbb{R}^n$, `TOL`, `MaxIters`, $A \in \mathbb{R}^{n \times n}$ (real & symmetric)

OUTPUT: $\lambda^{(k)}$ (approximation to λ_1 after k iterations)

$\mathbf{v}^{(0)} = \frac{\mathbf{v}^{(0)}}{\|\mathbf{v}^{(0)}\|}$ and $\lambda^{(0)} = \left[\mathbf{v}^{(0)}\right]^T A\mathbf{v}^{(0)}$

`mstop = 0` and $k = 0$

while `mstop==0` **do**
  $k = k + 1$

  $\mathbf{w} = A\mathbf{v}^{(k-1)}$, $\mathbf{v}^{(k)} = \frac{\mathbf{w}}{\|\mathbf{w}\|}$, and $\lambda^{(k)} = \left[\mathbf{v}^{(k)}\right]^T A\mathbf{v}^{(k)}$

  if $|\lambda^{(k)} - \lambda^{(k-1)}| < \texttt{TOL}$ **or** $k == \texttt{MaxIters}$ **then**
    $\texttt{mstop} = 1$
  end if
end while

Lab assignment: Matrix struct Applications

- Create matrix.c and matrix.h for linear algebra subroutines, including Gaussian Elimination with partial pivoting and Back Substitution for solving a linear system
- Application: 1D Imaging Deblurring by solving $A\mathbf{x} = \mathbf{b}$, where $\mathbf{x}$ is the true signal (unknown), A is the blur operator and $\mathbf{b}$ is the observed blurred signal.
  - Blur model: Discrete Convolution with symmetric blur kernel: $b_i = \sum_{-k}^{k} w_j x_{i-j}$, $\mathbf{w} = [1, 2, 1]/4$, so $b_i = (x_{i-1} + 2x_i + x_{i+1})/4$. Assuming zero boundary condition, this defines the linear system with
  $$A = \frac{1}{4}\begin{bmatrix} 2 & 1 & 0 & \cdots \\ 1 & 2 & 1 & \cdots \\ 0 & 1 & 2 & \cdots \\ \vdots & & & \ddots \end{bmatrix}$$
  - Generate Test Signal: $x_i = 1$ for $n/4 \le i \le n/2$, and 0 else. And compute $\mathbf{b} = A\mathbf{x}$.
  - Add noise to the observed signal $\mathbf{b}$ as $b_i = b_i + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$. Test with $\sigma = 10^{-6}, 10^{-4}$ and 10^{-2}.
  - Solve $A\mathbf{x}_{rec} = \mathbf{b} + \epsilon$, and measure the error $\|\mathbf{x} - \mathbf{x}_{rec}\|/\|\mathbf{x}\|$.
  - With Tikhonov regularization: $A^T A + \lambda I)\mathbf{x} = A^T\mathbf{b}$, with $\lambda = 10^{-6}, 10^{-4}, 10^{-2}$, also measure errors.
- Test with $n = 64, 128$.